Memoria Virtual

Roberto Giordano Lerena

Figuras tomadas de Stallings y de Silberschatz ©

Agenda

- ✓ Memoria Virtual.
- ✓ Falla de Acceso. Intercambio. Tipos de memoria. Técnicas de Paginación y Segmentación por demanda.
- ✓ Paginación por demanda.
  - Asignación y reemplazo de páginas. Criterios local y global. Páginas limpias y sucias.
  - Algoritmos de selección de la página víctiva (FIFO, MU, LU, Óptimo, LRU).
  - Anomalía de Belady. Hiperpaginación. Trashing.

Memoria Virtual

Hasta el momento, todas las técnicas de administración de memoria vistas respetaban el **Principio de Completitud** (el proceso debía estar cargado completo en memoria). La Memoria Virutal viene a romper con ese principio.

Memoria Virtual es una técnica de administración de memoria que permite la ejecución de procesos no completamente cargados en la memoria física.

Memoria Virtual - Condiciones

- ✓ Todas las referencias a la memoria serán direcciones lógicas que se traducirán dinámicamente a direcciones físicas durante la ejecución (capacidad de relocación dinámica).
- ✓ Un proceso puede dividirse en varias partes y no es necesario que todas estas partes se encuentren en la memoria principal durante la ejecución:
  - No será necesario que todas las páginas o todos los segmentos de un proceso estén en la memoria durante la ejecución.

Ventajas de la Memoria Virtual

- ✓ Se cargan sólo algunos fragmentos de un cada proceso. No se cargan los que no se requieren.
- ✓ Se pueden mantener más procesos en la memoria principal (Mayor N).
  - Con tantos procesos en la memoria principal es muy probable que uno de los procesos esté en estado Listo en un instante determinado.
- ✓ Es posible ejecutar procesos más grandes que toda la memoria principal. El Tamaño máximo de proceso es prácticamente infinito.

Fallo de acceso

- El sistema operativo crea el proceso y comienza trayendo sólo unos pocos fragmentos de su código a memoria.
- El “conjunto residente” es la parte del proceso que está realmente en la memoria principal.
- Si el procesador encuentra una dirección lógica que no está en la memoria principal, genera un **Fallo de acceso a memoria**, que provoca una interrupción.
- Un **Fallo de Acceso es una referencia válida a una dirección ausente en memoria.**

Fallo de acceso - Atención

- Ante un Fallo de acceso, el SO necesita traer a la memoria principal el fragmento del proceso que contiene la dirección lógica válida y ausente que provocó el Fallo de acceso:
  - El SO emite una solicitud de Lectura de I/O al disco (donde está alojado lo que require) y pone al proceso en estado Bloqueado por I/O.
  - El SO dispara otro proceso de usuario para que se ejecute mientras se realiza la operación de I/O.
  - Una vez que el fragmento requerido se ha traído a la memoria principal (se ha emitido la interrupción de satisfacción de la I/O), se devuelve el control al SO, y coloca el proceso afectado en el estado Listo.

Principio de cercanía (Localidad)

- ✓ Las referencias a los datos y al programa dentro de un proceso tienden a agruparse.
- ✓ Durante cortos periodos de tiempo se necesitarán sólo unos pocos fragmentos de un proceso.
- ✓ Sería posible hacer predicciones inteligentes sobre qué fragmentos de un proceso se necesitarán en un futuro cercano.
- ✓ El principio de cercanía sugiere que los esquemas de memoria virtual pueden funcionar eficazmente.

Tipos de memoria

- ✓ Memoria Principal

- ✓ Memoria Vitual (Memoria secundaria)
  - Memoria situada en el disco (área de MV o área de Swapping).
  - Permite una multiprogramación más eficiente.

Técnicas de Gestión de MV

- ✓ Paginación por demanda
  - Idem Paginación pura, pero no todas las páginas del proceso están cargadas en la memoria principal.
  - Las páginas se van trayendo a Memoria Principal cuando se las require.
  - Falla de página.

Técnicas de Gestión de MV

- ✓ Segmentación por demanda
  - Idem Segmentación pura, pero no todos los segmentos del proceso están cargados en la memoria principal.
  - Los segmentos se van trayendo a memoria principal cuando se los require.
  - Falla de segmento.

Técnicas de Gestión de MV

- ✓ Tiene que existir un soporte ad hoc de hardware para implementar la paginación/segmentación por demanda.
- ✓ El SO debe incluir un software para gestionar el movimiento de páginas o segmentos entre memoria secundaria y memoria principal.
- ✓ El SO debe implementar algorítmica ad hoc para lograr un uso eficiente de la memoria usando estas técnicas.

Paginación por Demanda (PxD)

- ✓ La totalidad de las páginas del proceso están en Memoria Secundaria.
- ✓ Cada proceso tiene asignados un conjunto de frames para operar.
- ✓ Algunas páginas del proceso están cargadas en Memoria Principal.
- ✓ Página que no se require, no se carga nunca.

Paginación por Demanda (PxD)

- A medida que se van produciendo Fallos de página, el SO va cargando las páginas que el proceso require en los frames que tiene asigandos, conforme las va solicitando (swap in).
- Cuando un proceso tiene todos sus frames asignados ocupados y se produce un Fallo de página, se requiere el reemplazo de alguna de las páginas cargadas, por la nueva requerida.
- En el reemplazo, primero debe descargar (swap out) la página que reemplazará (página víctima).

Indicadores

- ✓ Cada fallo de página significa un proceso de intercambio (swapping) que cuesta caro (tiempo de transferencia).
- ✓ Además, el proceso current pierde el procesador (context switch) hasta recuperar la página que require (tiempo de Bloqueado por I/O); recién ahí, vuelve a estado Listo.
- ✓ Ergo, hay que minimizar la **Cantidad de Fallos de página** y la Tasa de Fallos de página (cantidad de FdP en el tiempo).

PxD – Tabla de páginas

- ✓ Cada proceso tiene su propia tabla de páginas.
- ✓ Cada entrada de la tabla de páginas contiene el número de frame correspondiente en la memoria principal.
- ✓ Se necesita agregar un bit adicional a cada entrada en la tabla de cada proceso para indicar si la página en cuestión está presente en la memoria principal o no lo está.

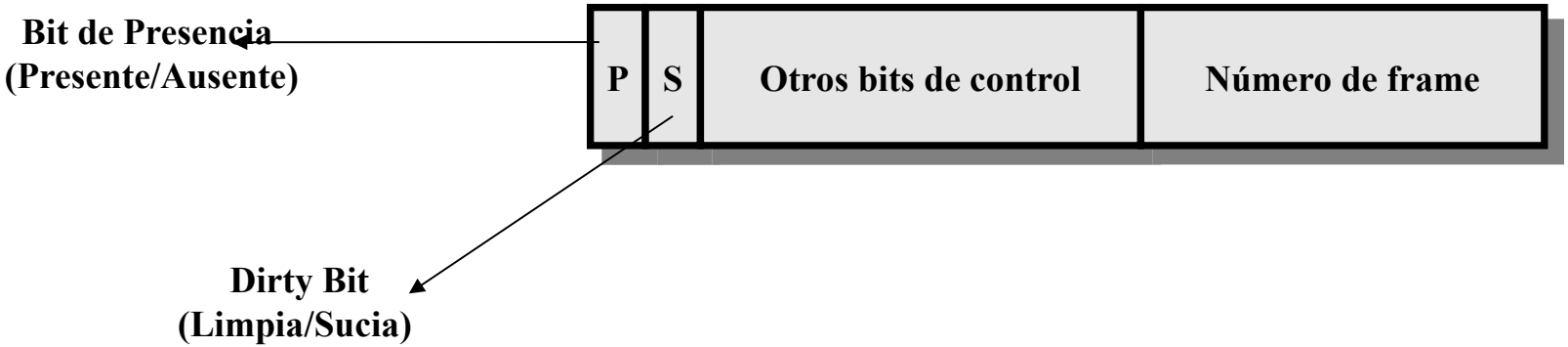
Páginas limpias y sucias

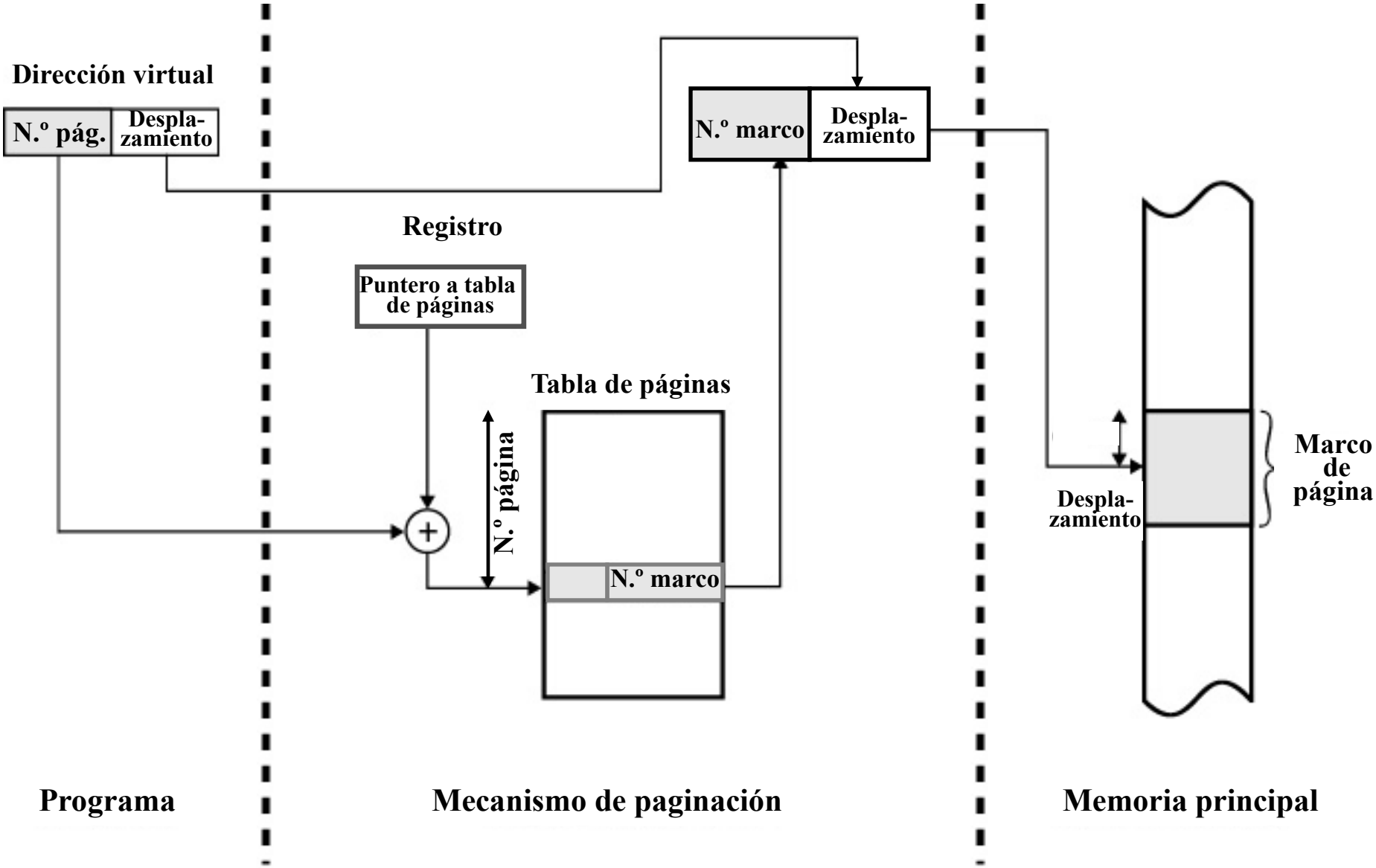
- La descarga (swap out) de la pagina víctima que se va a reemplazar se require para actualizar su imagen en memoria secundaria.
- Si la página víctima no se modificó (página limpia), no es necesario descargarla.
- Necesitamos, entonces, otro bit de control en cada entrada de la tabla de páginas, llamado Dirty Bit (bit de suciedad) que indica si la página se ha modificado desde fue cargada en la memoria principal.

Entradas de la Tabla de Páginas

Dirección virtual

Entrada de la tabla de páginas




Dirección virtual
N.º pág.
Desplazamiento
Registro
Puntero a tabla de páginas
Tabla de páginas
N.º página
N.º marco
N.º marco
Desplazamiento
Desplazamiento
Marco de página
Programa
Mecanismo de paginación
Memoria principal

Tablas de páginas

- La tabla de páginas completa puede ocupar una cantidad enorme de memoria principal.
- Para solucionar este problema, las tablas de páginas se almacenan en la memoria virtual.
- Cada referencia genera, entonces, dos accesos a la memoria virtual:
  - Uno para obtener la entrada de la tabla de páginas correspondiente.
  - Otro para obtener el dato deseado.
- Es muy poco perfomante => TdP del SO…

Tabla de Páginas SO (TLB)

- ✓ Como la Tabla de Páginas del SO tiene una cantidad finita de entradas, entonces hay que optimizar su uso.
- ✓ Se la utiliza como un cache, para guardar algunas entradas de la tabla de páginas:
  - Se denomina buffer de traducción adelantada (*TLB*, *Translation Lookaside Buffer*).
  - Contiene aquellas entradas de la tabla de páginas usadas hace menos tiempo (las mas recientemente usadas).

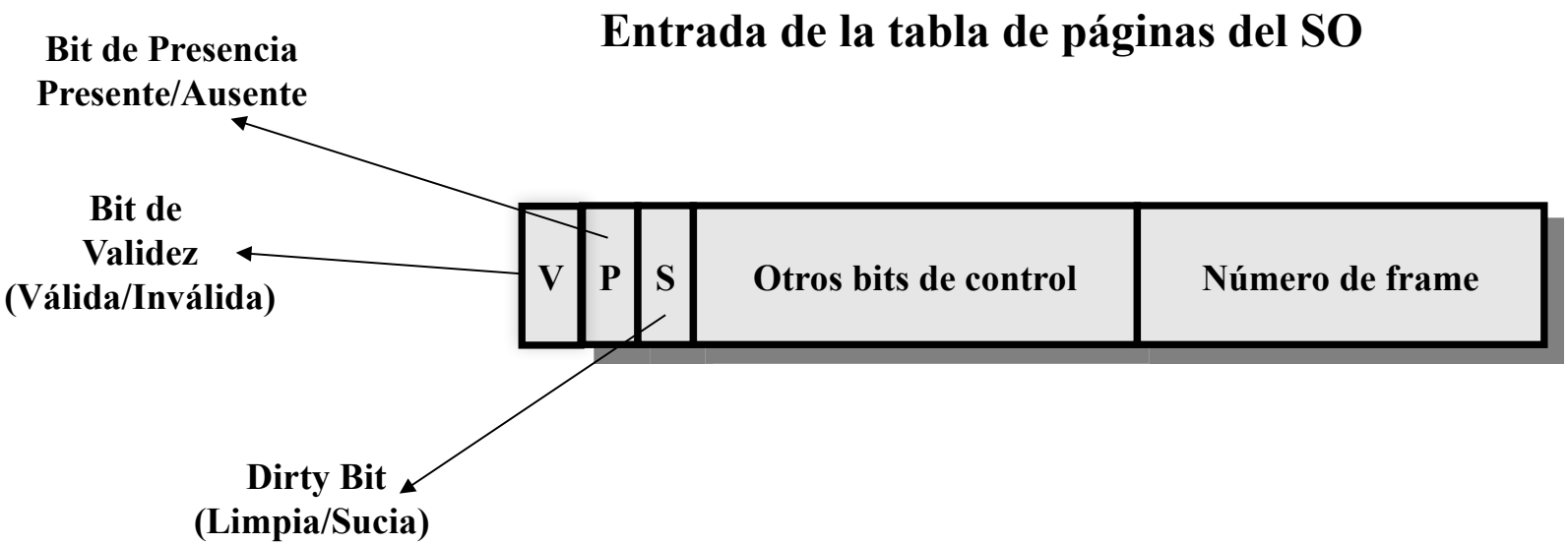
Tabla de Páginas SO (TLB)

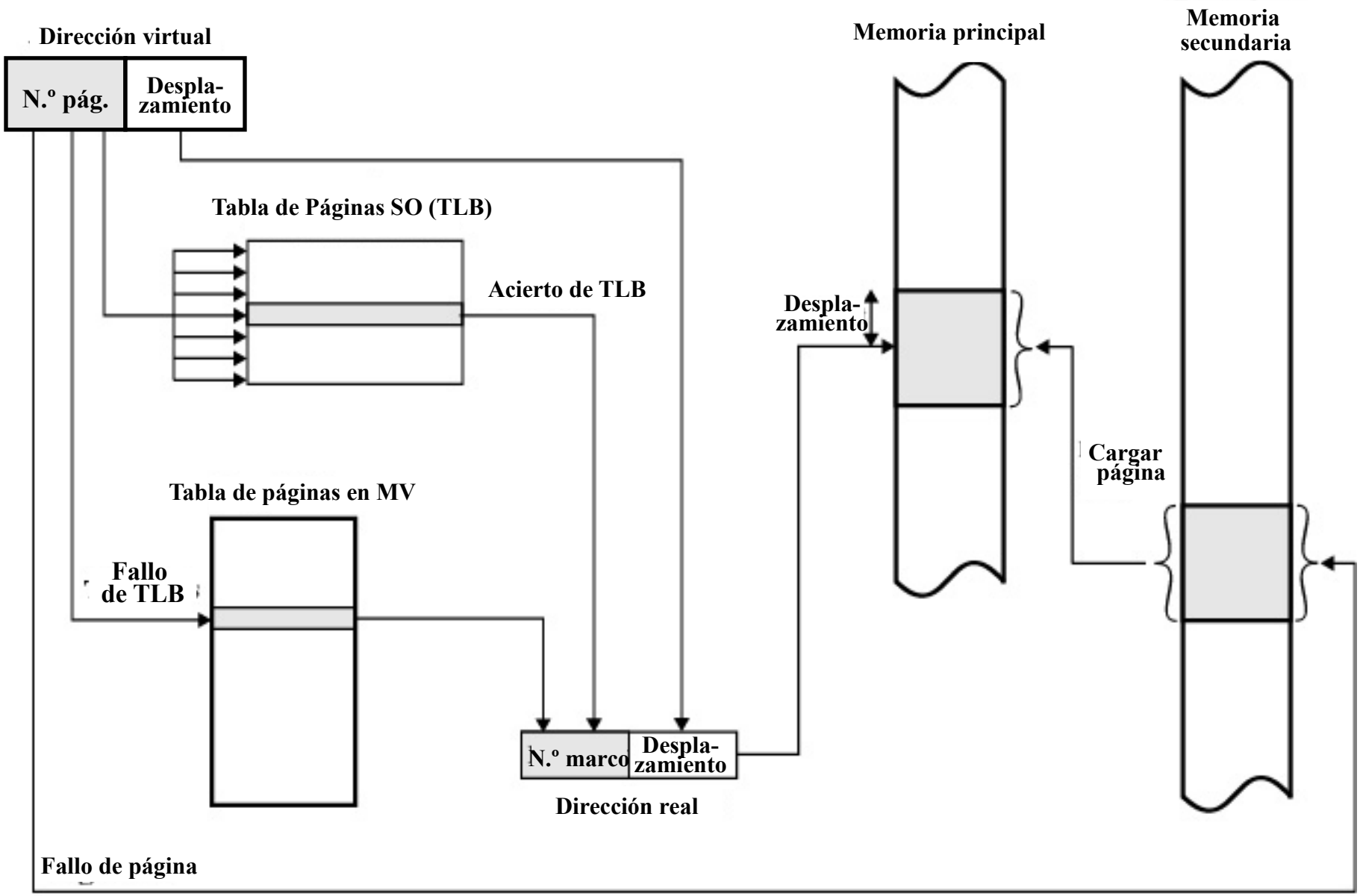
- ✓ Dada una dirección virtual, el procesador examinará primero la TLB.
- ✓ Si la entrada de la tabla de páginas buscada está presente (un “acierto en la TLB”), se obtiene el número de marco y se forma la dirección real.
- ✓ Si la entrada de la tabla de páginas no se encuentra (un “fallo en la TLB”), el procesador emplea el número de página como índice para buscar en la tabla de páginas del proceso en la memoria virtual, traer la entrada a la TLB y luego conformar la dirección física en memoria.

Tabla de Páginas SO (TLB)

- ✓ Conocida la entrada correspondiente a la página referenciada, se comprueba que la página propiamente dicha esté cargada en la memoria principal.
- ✓ Si está presente, se ejecuta la referencia.
- ✓ Si está ausente (Fallo de página), se debe cargar la pagina en memoria principal.

Entradas de la Tabla de Páginas





Direccionamiento en MV (Con TLB)

Tamaño de página

- ✓ Cuanto menor sea el tamaño de página, menor será la cantidad de fragmentación interna, y menor el tiempo de swapping.
- ✓ Cuanto menor sea la página, mayor será el número de páginas que se necesitan por proceso.
- ✓ Un número mayor de páginas por proceso significa que las tablas de páginas serán mayores (más entradas).
- ✓ Esto puede significar que una gran parte de las tablas de páginas de los procesos activos deben estar en la memoria virtual.

Políticas de asignación de frames

- ✓ Equitativa
  - Se asigna a todos los procesos la misma cantidad de frames
- ✓ Proporcional
  - Se asigna a cada proceso la parte proporcional de frames de la memoria principal, en función de su tamaño.
  - Procesos más grandes tendrán mayor cantidad de frames asignados. Procesos chicos, menor cantidad de frames asignados.

Políticas de asignación de frames

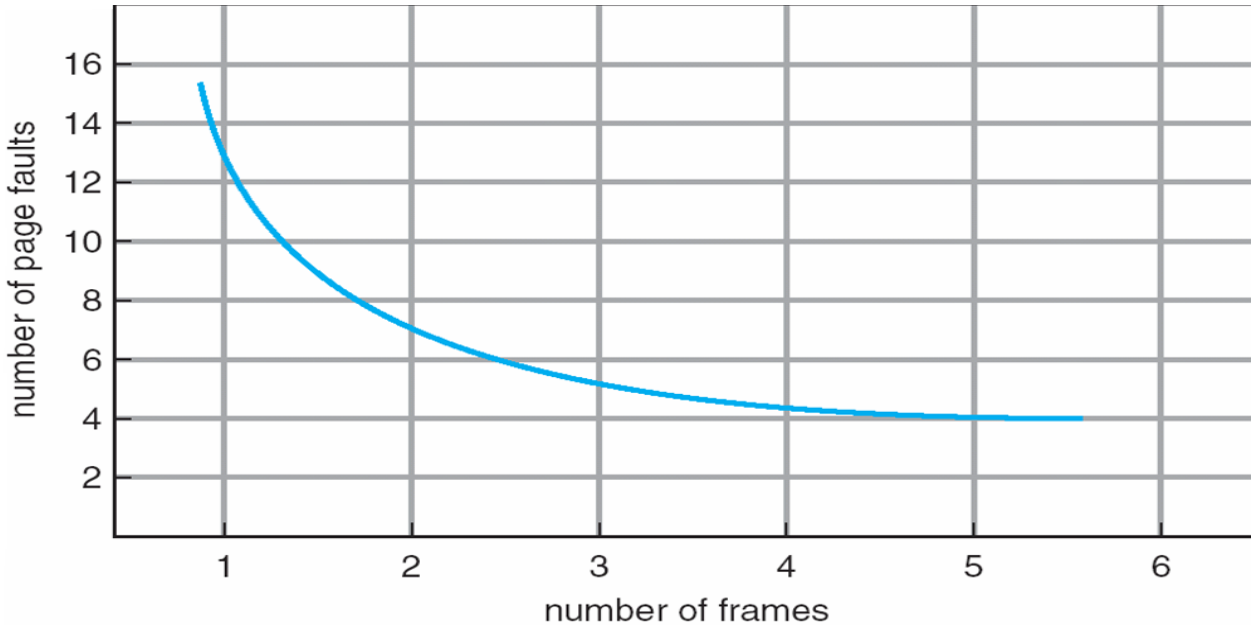
- ✓ Hay un número de frames mínimo que garantiza que el proceso pueda ejecutar una instrucción completa.

Sum A B C

Es la cantidad de operandos explícitos del asembler del procesador + 1

Políticas de asignación de frames

- ✓ En principio, cuanto mayor es la cantidad de frames asignados a un proceso, menor debería ser la cantidad de Fallos de página.



Políticas de carga inicial de páginas

- ✓ Prepaginación
  - Al inicio del proceso al inicio se cargan algunas se sus páginas en los frames asignados.

- ✓ Demanda estricta
  - Se cargan las páginas a medida que se van referenciando. Página que no es referenciada, no se carga nunca.

Criterios de reemplazo de páginas

- Criterio Local
  - Se reemplaza una página del conjunto de frames asignado al proceso.

- Criterio Global
  - Se reemplaza una página del conjunto total de frames de la memoria, sin importar a qué proceso está asignado.

Reemplazo - Selección de la víctima

- ✓ Ante la necesidad de reemplazar una página, se debe elegir cúal reemplazar (página víctima) tratando de elegir aquella que menor efecto tenga en la tasa de fallo de paginas.
- ✓ No todas las páginas son victimas candidatas. Hay algunas “páginas bloqueadas” (Las del núcleo del SO, Estructuras de control, Buffers de I/O, etc). El bloqueo se consigue asociando un bit de bloqueo a cada marco.

Selección de la víctima - Algoritmos

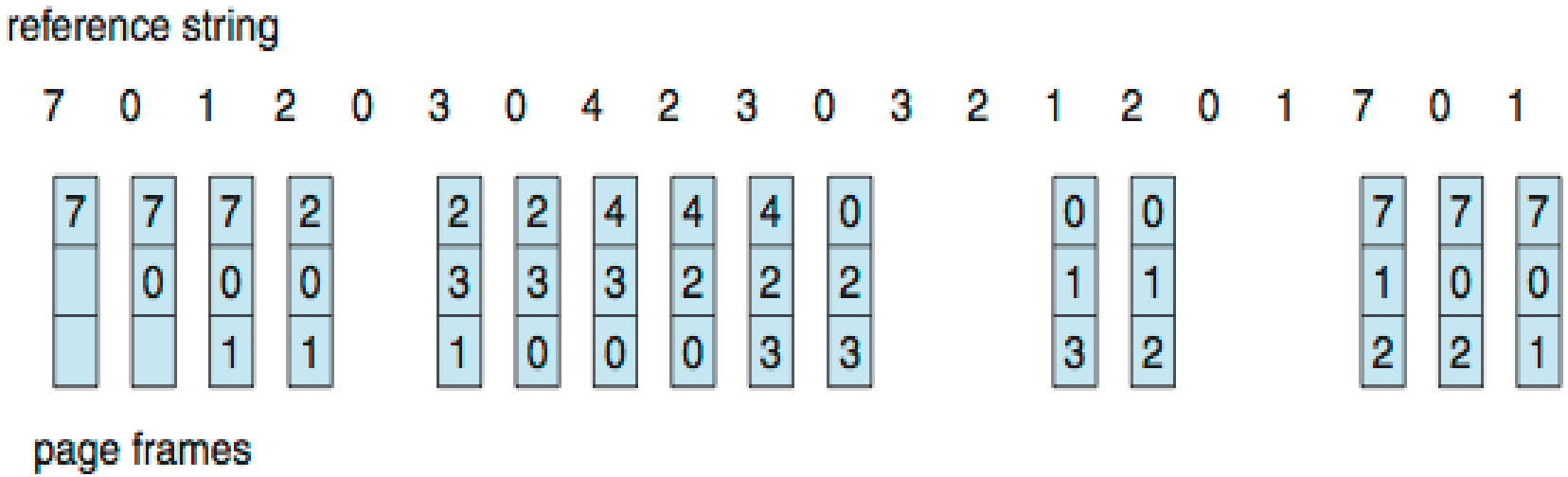
Primera en entrar primera en salir (FIFO)

- ✓ Se reemplaza la página que ha estado más tiempo en la memoria; la primera que se cargó.
- ✓ Requiere agregar un timestamp de tiempo de carga en la TP para las páginas presentes, que se actualiza en cada swap in.
- ✓ Es muy fácil de implementar.
- ✓ Estas páginas pueden necesitarse de nuevo y en un plazo de tiempo corto.

Algoritmo FIFO (ejemplo)

Secuencia de referencias: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**

3 frames asignados al proceso (solo 3 páginas puede tener en memoria)



Total: 15 Fallas de página

FIFO - Anomalía de Belady

Cuando aplicamos un algortimo FIFO de reemplazo, puede pasar que, incrementar los frames asignados a un proceso, en vez de reducir su cantidad de Fallos de página, los incremente.

- ✓ Esto se conoce como **<u>Anomalía de Belady</u>: Fenómeno que ocurre cuando incrementar los frames asignados a un proceso no garantiza la disminución de su cantidad de fallas de página.**

Selección de la víctima - Algoritmos

Más Usada (MU) y Menos Usada (LU)

- ✓ Se reemplaza la página que ha sido más/menos usada desde que se cargó.
- ✓ Requiere agregar un Contador de referencias en la TP para las páginas presentes, que se actualiza en cada vez que se la referencia.
- ✓ Es muy fácil de implementar.
- ✓ El mayor o menor uso de las páginas no tiene que ver con su uso futuro.

Selección de la víctima - Algoritmos

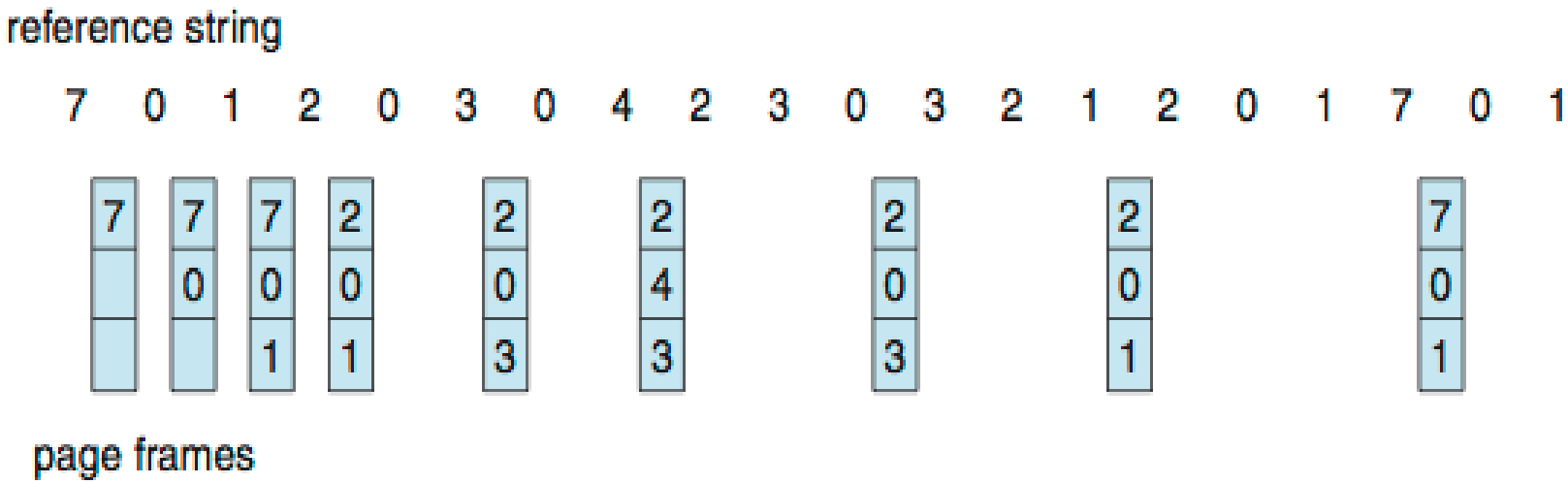
Algoritmo Optimal

- ✓ Selecciona para reemplazar la página que tiene que esperar una mayor cantidad de tiempo hasta que se produzca la siguiente referencia a la misma.
- ✓ Es imposible de implementar porque requiere que el SO tenga conocimiento de las referencias futuras… y el futuro es impredecible !
- ✓ Sirve como referencia para benchmarking

Algoritmo Optimal (ejemplo)

Secuencia de referencias: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**

3 frames asignados al proceso (solo 3 páginas puede tener en memoria)



Total: 9 Fallas de página (valor óptimo, mínimo)

Selección de la víctima - Algoritmos

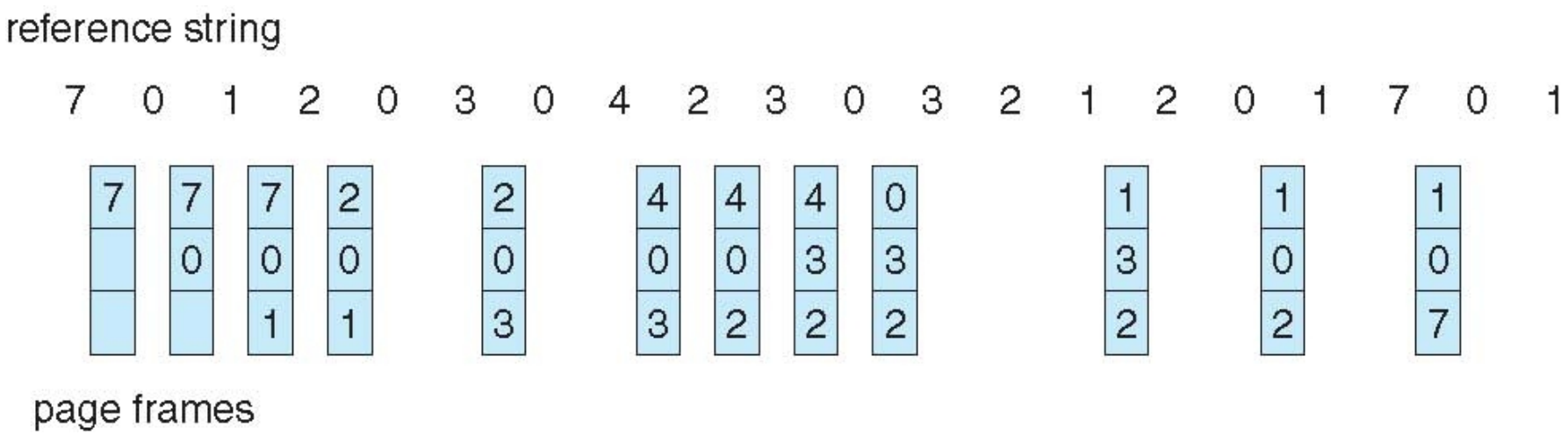
Menos recientemente usada (LRU)

- ✓ Reemplaza la página de memoria que no ha sido referenciada desde hace más tiempo.
- ✓ Debido al principio de cercanía, ésta sería la página con menor probabilidad de ser referenciada en un futuro cercano.
- ✓ Requiere agregar un timestamp de tiempo de última referencia en la TP para las páginas presentes, que se actualiza en cada referencia.

Algoritmo LRU (ejemplo)

Secuencia de referencias: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**

3 frames asignados al proceso (solo 3 páginas puede tener en memoria)



Total: 12 Fallas de página

Reemplazo de página - Proceso

- Fallo de página
- Se bloquea la página que hace la referencia
  - Si hay frames asignados libres: se asigna uno
  - Si no hay frames asignados libres: se determina el conjunto de víctimas candidatas (CVC)
    - Si la política de reemplazo es local, entran en el CVC sólo las páginas del proceso cargadas en memoria
    - Si la política de reemplazo es global, entran en el CVC todas las páginas cargadas en memoria (de todos los procesos)
  - Se reduce el CVC al conjunto de páginas limpias (si las hubiera)
  - Se aplica el algoritmo para elegir la página víctima dentro de las que integran el CVC (FIFO, LRU, etc)
- Swap Out de la página víctima (si fuera sucia)
- Swap in de la página ausente referenciada

Hiperpaginación

- Cuando tenemos una cantidad fija de frames asignados, reemplazo local y alta tasa de falla de páginas, decimos que estamos en Hiperpaginación, que es circunscripta al proceso en cuestión.
- El SO expulsa una página de un proceso, justo antes de ser usada.
- El procesador consume más tiempo intercambiando páginas del proceso que ejecutando instrucciones de usuario.

Trashing

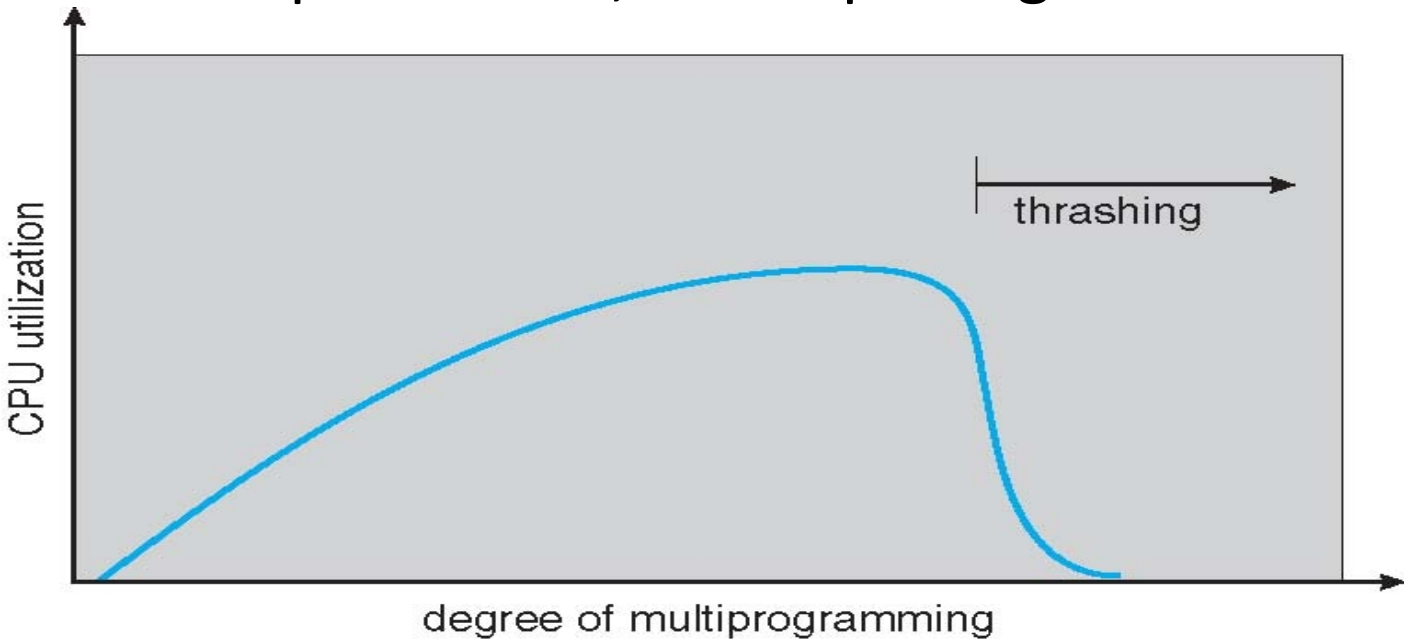
- ✓ Cuando tenemos una cantidad variable de frames asignados (no hay mínimo garantizado), reemplazo global y alta tasa de falla de páginas, decimos que estamos en Trashing, que afecta a todo el Sistema.
- ✓ El SO reemplaza una página de otro proceso, que luego ese proceso requerirá.
- ✓ Los procesos se van robando páginas entre si y ninguno puede ejecutar operaciones. Sólo hay fallos de páginas.

Trashing

- En un Trashing, el % de Uso efectivo del procesador tiende a 0, el Tiempo de Turnaround y de espera de los procesos tiende a infinito, la Productividad cae a 0 y el % de uso de los dispositivos de I/O está al 100%.
- El Trashing es un fenómeno indesable que afecta al Sistema de Computación en su conjunto, y provoca que el SO ocupe su tiempo en context switch y swapping más que en ejecutar instrucciones de procesos de usuarios.

Trashing

- ✓ El Schedulling de Memoria aumenta el grado de multiprogramación, inyectando procesos al Sistema (cuando el % de Uso efectivo del procesador es muy bajo).
- ✓ Cuanto más procesos inyecta, menor es el % de Uso efectivo del procesador, hasta que llega a 0.



Segmentación por Demanda

- ✓ Valen los mismos principios de gestion que en la paginación por demanda.
- ✓ Facilita la compartición entre procesos (es más probable compartir segmentos que procesos)
- ✓ La referencia sigue siendo por un par (Segmento, Desplazamiento)
- ✓ La Tabla de segmentos es más cara (más ancha)

Gracias !